A graphRAG-based framework for automated, robot-executable task planning using BIM knowledge graphs and large language models

Aditya Tandon, Munib Mahadik, Sudarsan Reddy Katam, Alexander Chmelnizkij, Kay Smarsly
Institute of Digital and Autonomous Construction, Hamburg University of Technology, Hamburg, Germany

Corresponding author: aditya.tandon@tuhh.de

Abstract—Robotic inspection and construction in the built environment increasingly require scalable task-planning methods that derive executable robot behavior from structured infrastructure information. Building information modeling (BIM) offers rich semantic, geometric, and relational representations of building elements and spaces, and therefore offers a foundation for robotic task planning. However, BIM models do not directly yield robot-executable control structures, creating a methodological gap between BIM-derived semantic knowledge and formal models for robot planning and execution. Existing research addresses isolated aspects of the problem, including BIM-driven planning, symbolic task decomposition, and behavior tree (BT)-based execution, but a unified framework that connects BIM semantics to executable robotic task plans is lacking. This paper proposes a graph retrieval-augmented generation (graphRAG)-based automated task-planning framework that leverages BIM knowledge graphs (KGs), large language models (LLMs), hierarchical task networks (HTNs), and BTs for robotic inspection workflows. BIM models are first transformed into KGs and queried through an LLM-based graph retrieval mechanism to obtain task-relevant semantic and spatial context. Based on the retrieved context, a rule-based HTN planner decomposes high-level inspection tasks into ordered primitive tasks, which are subsequently mapped to BT subtrees for robot execution. The framework is validated in BIM-based inspection scenarios. Graph retrieval is tested on three BIM models, and task decomposition and BT transformation on a topological map. Query translation passes 96.2 % of 53 queries, infeasible tasks are reported explicitly, and the generated BTs export to BehaviorTree.CPP v4 XML. The results indicate the feasibility of automatically generating semantically grounded, robot-executable task plans and highlight the potential of the proposed framework to support scalable robotic inspection and construction applications.

Keywords—Building information modeling; Knowledge graphs; Graph retrieval-augmented generation; Hierarchical task networks; Behavior trees; Product and process modeling

I. INTRODUCTION

Robotic inspection is gaining increasing relevance in the construction industry as robotic systems can improve the safety, efficiency, and repeatability of infrastructure inspection and monitoring processes [1]. Effective robot deployments require task representations that support both high-level semantic reasoning and reliable robot execution [2]. Inspection requirements must therefore be derived from structured information about the built environment, including elements requiring inspection, spatial context, and inspection objectives. Furthermore, inspection requirements must be translated into executable task structures that can be interpreted by robot task control architectures [3]. The gap between semantically grounded task reasoning and robot-executable task structures remains a central challenge for robotics in digital building environments. Consequently, an important research question concerns how to ground robotic inspection task planning in BIM-derived semantic and spatial information (RQ1).

Building information modeling (BIM) provides structured semantic, geometric, and relational representations of infrastructure and therefore offers a promising foundation for the automated derivation of robotic inspection tasks. However, transforming BIM-based information into robot-executable task plans requires reasoning mechanisms that are capable of interpreting relationships among building elements, spaces, inspection targets, and task-related constraints. Recent advances in large language models (LLMs) have demonstrated strong capabilities in semantic reasoning and structured generation, indicating substantial potential for deriving inspection tasks from structured infrastructure knowledge [4]. Nevertheless, generated plans must remain interpretable, modular, and executable for robotic systems [5]. A systematic transformation is therefore required to link high-level semantic task descriptions to execution-oriented task-control structures aligned with robot capabilities. In relation to BIM-grounded reasoning, another key challenge concerns how graph-based retrieval mechanisms can identify and extract task-relevant context from BIM-derived knowledge graphs (KGs) for robotic task planning (RQ2).

Existing research has addressed BIM-informed robotic inspection planning, symbolic task planning, and robot-executable task-control structures, yet the individual aspects have largely been treated as separate methodological layers [6]. In BIM-to-robot workflows, BIM has supported localization, navigation, inspection mission planning, and construction progress monitoring [7]. Symbolic planning approaches provide established mechanisms for representing goals, decomposition, and logical dependencies [8], whereas robot execution has frequently been implemented using modular, reactive behavior trees (BTs) [9]. Despite the aforementioned advances, the end-to-end transformation of semantically rich BIM information into automatically generated, robot-executable task models remains limited. In particular, research still lacks unified frameworks that can (i) retrieve task-relevant context from BIM-derived knowledge representations, (ii) generate hierarchical task structures from such context, and (iii) transform generated structures into executable control models for robotic inspection. Accordingly, a further research question concerns how retrieved BIM context can be transformed into hierarchical task structures, such as hierarchical task networks (HTNs) and

BTs, for robot execution (RQ3).

To address the identified gap, this paper proposes a graph retrieval-augmented generation (graphRAG)-based automated task-planning framework that utilizes BIM KGs, LLMs, HTNs, and BT-based execution models. Within the proposed approach, BIM models are transformed into KGs that capture semantic entities, properties, and relationships relevant to inspection tasks. A graphRAG mechanism, in which an LLM translates natural-language task descriptions into graph queries, subsequently retrieves task-relevant subgraphs from the BIM KG, thereby grounding task plans in building-specific semantic and spatial context. Based on the retrieved graph context, inspection tasks are decomposed using an HTN, which represents the task plan hierarchically in terms of subtasks, dependencies, and ordering relations. Task decomposition using HTNs is performed using rule-based methods and is therefore deterministic. HTN structures are subsequently mapped to BT-based execution models composed of reusable robot-capability subtrees. By integrating graph retrieval, HTN-based task decomposition, and execution-oriented BT synthesis, the framework connects BIM-derived semantic and spatial context with robot-executable task-control structures and establishes a unified pipeline for automated robotic inspection planning. The main contributions of this paper are (i) a graphRAG-based framework for retrieving task-relevant semantic and spatial context from BIM-derived KGs, (ii) an HTN-based task decomposition mechanism that generates HTN task structures grounded in retrieved BIM context, and (iii) a transformation approach that maps HTN task structures to BT-based execution models composed of reusable robot-capability subtrees.

The remainder of the paper is organized as follows. Section II presents the proposed graphRAG-based framework, including the BIM-to-KG transformation, the graphRAG retrieval process, HTN-based task decomposition, and the transformation from HTNs to BTs. Section III describes the validation setup, validation criteria, and results obtained in BIM-based inspection scenarios. Section IV concludes the paper with a summary of the main findings, limitations, and directions towards future research.

II. A GRAPHRAG-BASED FRAMEWORK FOR AUTOMATED, ROBOT-EXECUTABLE TASK PLANNING

This section presents the proposed automated task-planning framework using graph retrieval-augmented generation. The framework links BIM-derived semantic building information with executable robotic control structures through four transformation steps. First, BIM models are transformed into KGs represented as labeled property graphs. Second, task-relevant semantic and spatial information is retrieved using a graphRAG pipeline, in which natural-language task queries are translated into graph queries using an LLM-based Text2Cypher mechanism [10]. Third, the retrieved graph context is used for context-conditioned task decomposition, where an HTN planner applies predefined decomposition methods to expand high-level inspection tasks into ordered subtasks and primitive actions. The decomposition is grounded in BIM-derived spatial and semantic context and is constrained by the capabilities of the robot assigned the inspection task. Fourth, the primitive HTN tasks are linked to reusable robot-capability subtrees [1] and transformed into BT-based execution models for robotic task control. Figure 1 presents the proposed methodology as a four-stage workflow that connects BIM-based semantic retrieval with robotic task planning. The following subsections describe the BIM-to-KG transformation, graphRAG-based information retrieval and task grounding, HTN-based task decomposition from BIM KGs, and the HTN-to-BT transformation for robotic task control.

BIM-to-KG transformation

The BIM-to-knowledge-graph (BIM-to-KG) transformation represents the building model as a structured semantic graph that preserves both entities and relationships. Building elements, including spaces, walls, doors, windows, furniture, sanitary fixtures, equipment, and structural elements such as columns, beams, and slabs, are modeled as typed graph nodes, while element attributes are stored as node properties and element interconnections are encoded as graph relationships. The BIM model serves not only as a geometric description of the built environment but also as a semantic representation that supports robotic task planning and reasoning.

Within the proposed framework, the most relevant relationships that enable task grounding in the spatial and semantic context of the building include, in particular, element-to-space containment relationships and space-boundary relationships. Based on the relationships, door-based space-to-space transition relationships are derived by identifying configurations in which two spaces are connected through a common door, yielding a navigable connectivity graph. The relationships enable the retrieval of task-relevant building context and the projection of inspection tasks onto a navigable spatial structure that can subsequently be used for planning and execution.

The BIM-to-KG transformation workflow is implemented by parsing the Industry Foundation Classes (IFC) model with IFC-graph [11] and storing the resulting labeled property graph (LPG) in Neo4j [12]. The BIM-derived LPG is subsequently processed using a Python-based pipeline. Within the graph, IFC entities are represented as nodes with labels and attributes. IFC relationship objects, such as spatial containment and space-boundary relationships, are explicitly modeled as intermediate nodes. Directed edges between nodes represent object-attribute references through which relationship nodes are linked to the associated IFC entities. The BIM KG, as a result, serves as the common representation layer for subsequent graph-based retrieval, task grounding, and the generation of execution-oriented spatial subgraphs.

graphRAG-based information retrieval and task grounding

Building on the BIM KG, the framework employs a graphRAG retrieval layer to translate natural-language robot task requests into graph-grounded building information. The objective of the retrieval layer is not only to identify isolated building elements but also to retrieve spatially and semantically contextualized information required for robotic task planning. An inspection task must therefore be broken down into target elements, corresponding IFC classes, associated containing spaces, coordinates, and the relevant spatial connections required to reach the elements.

After the IFC model has been transformed into an LPG,

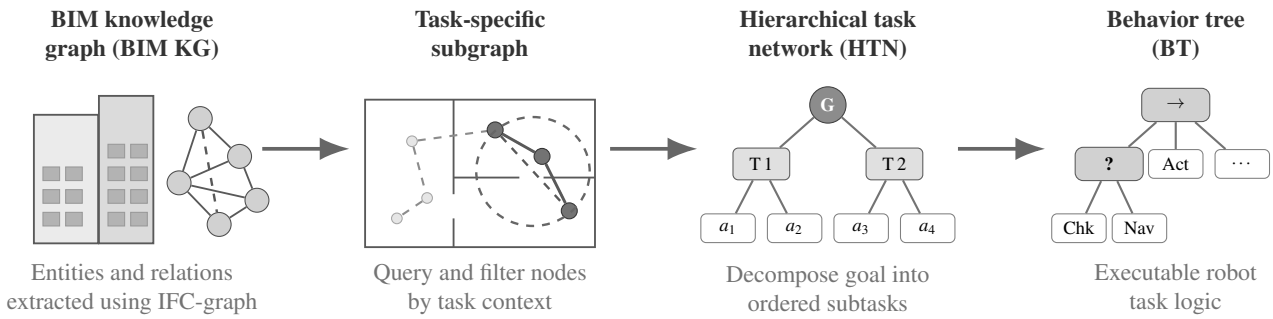


Figure 1. Four-stage methodology for BIM-based semantic retrieval and robotic task planning

the retrieval process is implemented through a schema-guided Text2Cypher mechanism. A natural-language task query is provided to an LLM together with the BIM graph schema and domain-specific few-shot examples, which encode how different task intents and object categories should be translated into Cypher queries. The schema guidance defines available node labels, properties, and relationship patterns, while the few-shot examples constrain the LLM toward valid graph traversals and task-relevant return fields. Constraints imposed by the examples reduce the risk of generating syntactically valid but semantically inappropriate queries, for instance, by querying non-existent IFC classes or omitting the spatial context required for robot execution.

The Text2Cypher component follows explicit retrieval rules that reflect the structure of IFC-derived graphs. Room names mentioned in natural language are matched against the human-readable *LongName* attribute of *IfcSpace*, while room numbers are matched against the Name attribute. Element retrieval considers both English and German naming variants, since BIM object names may originate from different authoring environments. Furthermore, object classes are selected according to task-relevant IFC storage conventions. For example, furniture-related task targets are queried as *IfcFurniture*, sanitary fixtures as *IfcFlowTerminal*, and equipment as *IfcBuildingElementProxy*. To reduce duplicate results caused by IFC relationship nodes, generated queries use distinct result sets when traversing undirected relationship patterns.

For robot task queries, the retrieval layer is designed to produce full-context records. Each retrieved task target is returned together with the identifier, the containing space, the containing-space identifier, and Cartesian coordinates. The standard return structure consists of the fields *name, id, space_name, space_id, x, y,* and *z*. The design of the return structure ensures that the output of information retrieval can be directly consumed by the subsequent task-grounding stage. In contrast to simple diagnostic queries, which may only list elements or counts, full-context task queries provide the minimum information required to instantiate robot navigation and interaction tasks. If a generated query returns only element names and identifiers without the associated containing spaces, the retrieval layer runs a backup query to recover which space each element belongs to. The use of a backup query allows task grounding to degrade gracefully rather than fail, returning an empty subgraph only when no containment information exists in the graph.

Following graph-grounded retrieval, the retrieved records are transformed into a task-specific spatial execution subgraph. The task-specific subgraph organizes the retrieved elements by containing spaces and attaches element-level coordinates where available. Each space in the task-specific subgraph contains the identifier, semantic name, optional space coordinates, and a list of associated task-relevant elements. The representation makes the retrieved information suitable for downstream task decomposition, since the planner can reason about which elements must be inspected, where elements are located, and how elements are spatially distributed across the building.

In addition to target elements and spaces, the framework reconstructs the relevant building space connectivity graph. A building-level door-space connectivity graph is extracted from the BIM KG using *IfcSpace*, *IfcRelSpaceBoundary*, and *IfcDoor* relationships. The connectivity graph represents spaces as connected through doors and is cached as a reusable topological representation of the building. For each task, the system identifies doors connected to the task-relevant spaces and stores door identifiers, names, widths, operation types, connection endpoints, and assumed initial states. The space connectivity graph representation enables the task-specific subgraph to represent not only where task targets are located, but also which doors and transitions may be relevant for reaching the task targets.

The framework then derives an approximate space-traversal sequence over the task-relevant spaces. A greedy multi-target breadth-first search is applied to the space connectivity graph, repeatedly expanding to the nearest unvisited task-relevant space, to generate an ordered sequence of spaces and doors connecting the spaces that contain task targets. The traversal sequence determines the order in which the compound inspection tasks are handed to the HTN planner, whereas the route between two consecutive spaces is recomputed by the planner during decomposition, as described in the following subsection. The traversal is not intended to replace metric path planning; rather, the traversal provides a semantic-topological ordering of spaces that is subsequently refined by a robot navigation stack.

The graphRAG layer is implemented in Python on top of Neo4j, with a locally served LLM (via Ollama) performing the Text2Cypher-based natural-language querying. Conditioned on the BIM graph schema and representative few-shot task-query examples, the model translates natural-language task descriptions into Cypher queries that retrieve task-relevant entities and the associated context from the BIM KG. The entities are then grounded through corresponding spaces, relevant door-based space-to-space transitions, and spatial connectivity and coordinate information, yielding a task-specific subgraph for

subsequent task generation. The task-specific subgraph contains execution-oriented information, such as task descriptions, spaces, elements, doors, and traversal sequences, and serves as the interface between BIM-based semantic retrieval and the later stages of robotic task planning.

HTN-based task decomposition from BIM KGs

Following graph-based retrieval and task grounding, the task-specific execution subgraph serves as the semantic and spatial basis for HTN-based task decomposition. Rather than generating task decompositions directly with an LLM, the proposed framework adopts a rule-based planning strategy in which inspection objectives are decomposed using predefined HTN operators and methods. The LLM is used only to retrieve and ground task-relevant information from the BIM KG, while task decomposition is performed by a deterministic planner. The separation ensures that the generation of robot-executable task structures remains controlled, reproducible, and explicitly constrained by the semantic information retrieved from the BIM model and by the capabilities of the assigned robot.

The task-specific subgraph provides planning-relevant representations of the building environment. The subgraph contains inspection targets, the associated spatial context, door-based connectivity between spaces, and semantic information required to determine suitable inspection actions. BIM-derived entities contained in the subgraph are interpreted as planning objects that define the scope and context of the inspection task. As a next step, inspection targets are associated with containing spaces and are assigned pre-determined inspection requirements based on semantic classification. For example, elements such as fixtures require visual inspection, wall segments, pipes, and ducts require thermal inspection, and columns or slabs require inspection with a depth camera. Inspection objectives are therefore not treated solely as abstract natural-language commands, but rather as a set of grounded task instances derived from the BIM KG.

Before task decomposition, the BIM-derived planning context is transformed into a world state that the HTN planner can interpret. The world state represents the relevant inspection elements, inspection status, the spaces in which inspection elements are located, the current state of doors, and the topological connectivity between spaces. The space connectivity graph provides a spatial abstraction of the environment, with spaces represented as nodes and doors as traversable connections. In addition, the robot is represented through a set of capability descriptions, including sensing capabilities such as RGB, thermal, and depth perception, as well as action capabilities such as manipulation and stair traversal. Robot capabilities determine whether robots are suitable for a given inspection task and whether robots are capable of traversing the spatial relations required to reach the target inspection elements.

The HTN decomposition process follows a top-down strategy in which high-level inspection goals are recursively refined into lower-level subtasks until only primitive robot actions remain. Each target inspection element is directly associated with a compound inspection task that must be decomposed into navigation, sensing, and result-recording activities. The choice of task decomposition method depends on the inspection modality required by the target inspection element and on whether the assigned robot possesses the corresponding sensing capability. For instance, a target requiring visual inspection can only be decomposed using a method that includes an RGB capture action, and the method is applicable only if the robot provides the required RGB sensing capability. Similarly, thermal and depth-based inspection tasks are decomposed through modality-specific methods that introduce the corresponding sensing actions.

Spatial task refinement is achieved by decomposing navigation-related compound tasks. To inspect a target, the robot must first reach the space in which the target is located. The BIM-derived space connectivity graph is therefore used to determine a valid route from the current space of the robot to the target space. The navigation route provides the ordering structure for navigation and access-related actions. Movement between connected spaces is represented as a navigation action, while closed doors along the route introduce additional door-manipulation actions. Traversal requirements encoded in the connectivity representation, such as movement between building levels, are evaluated against the capabilities of the robots, and the generated task structure therefore reflects both the spatial organization of the building and the practical constraints imposed by the robot platform. The planner does not assert task dependencies independently of the building model. Instead, dependencies emerge from the semantic and spatial requirements of the decomposed task. The task hierarchy resulting from the decomposition process therefore encodes not only the actions that must be performed, but also the order and the conditions under which actions must be performed.

The rule-based formulation also provides an explicit mechanism for identifying infeasible tasks. During decomposition, methods and primitive actions are evaluated with respect to the current world state and the capabilities of the robots. If a target cannot be reached through the available building connectivity, or if the assigned robot lacks a required sensing or traversal capability, the corresponding inspection task cannot be decomposed into an executable plan. In such cases, no valid HTN plan is returned for the task. Accordingly, infeasibility is treated as an explicit outcome of planning rather than an implicit failure during execution, which ensures that the generated HTN plan is structurally coherent, semantically grounded, and constrained by the actual execution context.

The output of the HTN stage is an ordered sequence of grounded primitive tasks. Each primitive task specifies the action to be executed, the robot responsible for execution, and the relevant task arguments, such as the target space, door, or inspection element. The sequence of obtained primitives therefore forms the interface between BIM-grounded semantic task reasoning and robot-executable control.

At the implementation level, the planner is implemented in Python without relying on an external planning library, and the HTN domain knowledge is explicitly encoded as an operator library and a method library. All primitive operators, *navigate*, *manipulate_door*, *capture_rgb*, *capture_thermal*, *capture_depth*, and *record_result*, are defined as functions that specify preconditions and effects on the world state, whereas compound tasks, namely *inspect_element* and *go_to*, are associated with an ordered list of candidate decomposition methods. Planning proceeds as a recursive total-order decomposition that expands tasks depth-first and simulates operator effects on a

copy of the world state, allowing the planner to discard partial expansions and backtrack to alternative methods whenever a branch becomes infeasible. The world state records robot positions, robot capability sets, element locations, inferred element types and inspection modalities, inspection status, and door states. The building connectivity graph is stored as an adjacency list in which each edge carries the door-manipulation action required and the capabilities needed for traversal. Routes between spaces are computed over the adjacency list using breadth-first search. Inspection modalities are assigned through a keyword-based classification of IFC element names that covers both English and German naming variants, and robot capabilities are derived from declarative robot-description files. The final plan is output in JSON format, capturing the ordered primitive tasks together with the initial robot and door states to provide a self-contained and reproducible record of the planning context.

In the subsequent stage, primitive HTN tasks are mapped to reusable robot-capability subtrees and assembled into a BT for autonomous robotic execution.

HTN-to-BT transformation for robotic task control

The HTN plan produced by the planner is a structured representation of inspection tasks. However, HTNs are primarily planning representations and are not directly executable by robotic systems. While HTNs specify which tasks must be performed and how tasks are further decomposed into subtasks, HTNs do not define the runtime control logic required for robotic task execution and control in unstructured and dynamic environments.

To ensure the generation of robot-executable task structures, HTNs are transformed into BTs, which provide a suitable execution model for robotic task control. Behavior trees provide modular and reactive control structures with reusable subtrees for autonomous robotic inspection. As shown in previous work from the authors [1], BT task control structures allow conditions and actions to be evaluated during runtime and support failure handling through explicit control-flow constructs.

The HTN-to-BT transformation step maps HTN concepts to BT constructs. Prior to mapping, the planner decomposes all compound tasks into a flat ordered plan of primitive actions, where each primitive HTN operator is mapped to a predefined, reusable BT subtree. Within the predefined subtrees, ordering relations are represented by sequence nodes, runtime checks by condition nodes, recovery and alternative behaviors by fallback nodes, and negations by decorator nodes. The individual operator subtrees are then assembled under a mission-level sequence node that preserves the order of the primitive HTN plan. Runtime checks, recovery, and alternative behaviors are handled within the predefined operator subtrees.

The HTN-to-BT transformation is implemented as a software module following HTN decomposition. The HTN plan consisting of task primitives is mapped to BT structures using a predefined operator-to-subtree library. The implementation is realized in Python and produces BehaviorTree.CPP v4 XML, compatible with the BehaviorTree.CPP-based Robot Operating System 2 (ROS 2) navigation stack. The custom navigation, sensing, and reporting leaf nodes referenced by the tree are registered with the execution node library at runtime. Primitive HTN tasks are linked to reusable robot-capability subtrees, such as navigation, door manipulation, inspection, and reporting behaviors. The BT resulting from HTN transformation serves as the executable control logic for robots, completing the transformation from BIM-derived semantic task planning to robotic task execution.

Building on the graphRAG-based framework presented in this section, the following section validates the generated task plans and BT structures using representative BIM-based inspection scenarios.

III. VALIDATION

Section II has introduced the graphRAG-based framework as a four-step methodology comprising BIM-to-KG transformation, graphRAG-based information retrieval and task grounding, HTN-based task decomposition from BIM KGs, and HTN-to-BT transformation for robot task control. In the validation, the BIM-to-KG transformation provides the BIM KG used as the basis for the subsequent tests. The validation therefore focuses on the three components that retrieve, generate, or transform task representations, i.e. (i) graphRAG-based information retrieval and task grounding, (ii) HTN-based task decomposition from BIM KGs, and (iii) HTN-to-BT transformation for robotic task control. The following subsections describe the validation objectives and datasets, the validation tests, and the validation results.

Validation objectives and datasets

The main objective of the validation is to assess whether the framework produces task representations that are semantically grounded, structurally valid, and execution-oriented, based on information derived from BIM. The validation uses three BIM models from a publicly available dataset as case studies, each transformed into a separate BIM KG. An additional scaled-down topological map, which is not derived directly from the IFC models, is used to test HTN-based task decomposition and HTN-to-BT transformation. Specifically, the validation assesses whether

1. the *graphRAG-based information retrieval and task grounding* process extracts semantic and spatial information relevant to robotic task planning from the BIM KG.

2. the topological and capability information represented in the scaled-down topological map and robot descriptions can be transformed through *HTN-based task decomposition* into valid HTN structures containing grounded tasks, ordering relations, and constraints.

3. HTN structures produced by the framework can be systematically transformed through *HTN-to-BT transformation for robot task control* into BT structures while preserving task dependencies and execution logic.

Validation tests

Three groups of validation tests are conducted, corresponding to the three methodological components of the framework. Each group of validation tests is assigned to the corresponding methodological component as follows.

- ***Information-extraction tests*** for graphRAG-based information retrieval and task grounding are conducted to validate whether the graphRAG layer and the Text2Cypher

model retrieve correct BIM and ontology information from natural-language queries. The tests utilize three IFC models from the IFC-Bench-v2 [13] dataset, an office building (fantasy_office_building_3), a hotel (fantasy_hotel_1), and a residential building (fantasy_residential_building_1), each transformed into a BIM KG. For each building, natural-language queries organized into seven levels of increasing semantic and spatial complexity are evaluated: Building-metadata counts, keyword-based element inventories, containment-based spatial hierarchy, element-coordinate retrieval, room-based queries filtered by human-readable space names, task-grounded full-context queries returning elements together with containing spaces and coordinates, and cross-language bridging between English and German element terms. Each generated Cypher query is executed against the BIM KG, and the returned records are compared against manually verified reference values. A query is counted as passed if the retrieved values match the reference graph values and no unsupported or non-existent entities are returned. To assess the effect of model capacity on schema-aware Cypher generation, every query is executed with two models of the qwen2.5-coder family [14], a 3-billion-parameter model and a 14-billion-parameter model, under identical schema guidance and few-shot prompting.

- ***HTN-decomposition tests*** for HTN-based task decomposition are conducted to validate whether the topological information represented in the scaled-down topological map and the robot capability information can be transformed into valid, executable task hierarchies. The validation is performed as a deterministic decomposition-and-simulated-execution test rather than as a generation task, i.e., the planner expands each compound task into primitive operators using predefined decomposition methods, the scaled-down topological map, the per-robot capability model, and the current world state. Each generated decomposition is checked against the primitive sequence implied by the grounded route and verified by simulated execution against the operator preconditions, using four criteria: completeness, correctness, ordering, and groundedness. The test for completeness is passed if the compound task decomposes fully into primitive operators. The test for correctness passes when the preconditions of every primitive hold when the primitive is executed and the simulated mission reaches the goal state. The test for ordering passes when the ordering imposed by the route and by action dependencies is preserved, e.g., a door is opened before the door is traversed. The groundedness test is passed when all referenced locations, elements, and capabilities originate from the scaled-down topological map, the robot descriptions, and the world state rather than from random or synthetic values. Last, the infeasibility test passes when cases, such as a destination unreachable under the capabilities of the robot, yield no plan and a planning failure is reported explicitly rather than being converted into unsupported robot actions.

- ***HTN-to-BT transformation tests*** for robot task control are conducted to validate whether HTNs generated by the framework can be transformed into structurally valid BTs. The validation is performed as a structural mapping test rather than as an experiment with physical robots. Primitive HTN tasks, such as navigation, door manipulation, sensor-based inspection, and result reporting, must be mapped to corresponding BT action nodes or reusable robot-capability subtrees, with each primitive operator associated with exactly one predefined subtree in a fixed BT library. As compound HTN tasks are fully decomposed into primitive operators by the planner before mapping, the BT structure is expressed as an ordered composition of capability subtrees assembled under a mission-level sequence node rather than as separately named higher-level subtrees. The test passes if the BT preserves the execution order defined in the HTN through sequence nodes, represents runtime-state preconditions as BT condition nodes before the corresponding action nodes, includes explicit failure or recovery branches for execution-time errors, and contains a valid root node, valid control-flow nodes, and leaf nodes that are registered in the library and executable.

Validation results

The results are reported for the three methodological components validated in this section.

For *graphRAG-based information retrieval and task grounding*, the information-extraction tests show that the framework retrieves building metadata, spatial hierarchy, element inventories, element coordinates, room-scoped properties, and bilingual concepts from the BIM KG. Across the three benchmark buildings, the system correctly returns task-relevant information, including room and door counts, element inventories such as furniture elements, containing-space relationships, element coordinates, and bilingual element mappings. As summarized in Table 1, over the 53 queries spanning the three buildings, the 3-billion-parameter model produces correct results for 44 queries (83.0%), with per-building pass rates of 71% (office), 94% (hotel), and 83% (residential). The 14-billion-parameter model produces correct results for 51 queries (96.2%), with per-building pass rates of 94% (office), 100% (hotel), and 94% (residential). The two failures of the larger model are element-inventory queries that return no records although the reference graph contains matching elements. The smaller model reproduces empty-result cases and exhibits two additional failure modes, occasionally generating syntactically invalid Cypher, through unbound variables or unbalanced parentheses, and omits required coordinate fields in queries that must return element coordinates. The results indicate that both model sizes are capable of handling building-metadata, containment, and cross-language queries, whereas higher model capacity primarily improves the validity of the generated Cypher and the completeness of coordinate retrieval required for robot task grounding. An example of a full-context robot-task query translated by the Text2Cypher model and executed against the office BIM KG (qwen2.5-coder:14B) is shown in Figure 2. The natural-language request “inspect all desks” is mapped to a schema-correct Cypher query that traverses element-to-space containment, bridges English and German element names, and returns each desk together with the containing space and world coordinates. The query passes, returning 79 records.

For *HTN-based task decomposition from BIM KGs*, valida-

```
//  Natural-language query: "inspect all desks"
MATCH (e:IfcFurniture)--(r:IfcRelContainedInSpatialStructure)--(s:IfcSpace)
WHERE toLower(e.Name) CONTAINS 'desk'
   OR toLower(e.Name) CONTAINS 'schreibtisch'
RETURN DISTINCT
  e.Name AS name,
  e.__step_file_id AS id,
  COALESCE(s.LongName, s.Name) AS space_name,
  s.__step_file_id AS space_id,
  e.world_x AS x, e.world_y AS y, e.world_z AS z
ORDER BY space_name, e.Name
```

Figure 2. Text2Cypher translation of the natural-language query "inspect all desks" into a schema-correct Cypher query returning each desk with the corresponding containing space and world coordinates (qwen2.5-coder:14B; passed, 79 records).

Table 1. Information-extraction performance of qwen2.5-coder 3B and 14B across three IFC-Bench-v2 buildings.

Building	Total queries	3B	3B %	14B	14B %
Office	17	12 of 17	71%	16 of 17	94%
Hotel	18	17 of 18	94%	18 of 18	100%
Residential	18	15 of 18	83%	17 of 18	94%
Total	**53**	**44 of 53**	**83.0%**	**51 of 53**	**96.2%**

tion is conducted over representative single-robot inspection-routing missions spanning two building levels and involving two robot platforms with different locomotion capabilities. The robots utilized for the tests include a wheeled mobile manipulator and a legged manipulator, each loaded from a capability description. As task decomposition is rule-based and grounded in the topological map and robot capability models, the task decomposition process is deterministic and reproducible, and all representative cases satisfy the four criteria. Completeness and ordering hold for all cases, i.e., every compound *go_to* task expands fully into a *navigate* or *manipulate_door* primitive sequence, where the task order follows the grounded route, and simulated execution of each plan reaches the goal state while satisfying the operator preconditions. A representative query involves the navigation of the legged robot starting at the ground-floor corridor and reaching and entering the first-floor room. The HTN planner finds a route over the topological map, inserts the required staircase-traversal and door-manipulation actions, and generates a grounded primitive sequence comprising staircase traversal, corridor navigation, and door manipulation. Simulated execution of this sequence reaches the target room. The same query for a wheeled robot, which lacks the stair-traversal capability, returns no plan rather than an erroneous stair-climbing action, demonstrating explicit handling of infeasible cases and capability grounding. The HTN-based task decomposition mechanism is additionally exercised on full inspection plans, in which each compound inspection task is decomposed into navigation, door-manipulation, sensing, and reporting primitives to achieve full element coverage. The remaining limitation lies upstream of decomposition: correctness at the level of fine-grained inspection requirements depends on the granularity of the BIM-to-graph extraction, since element type classification and sensor assignment are inferred from the graph and are only as detailed as the attributes the BIM KG affords.

For *HTN-to-BT transformation for robotic task control*, the mapping tests show that the HTNs generated by the framework can be systematically transformed into structurally valid BTs. All primitive HTN tasks, such as navigation, door manipulation, sensor-based inspection, and result reporting, map to a corresponding BT action node or reusable robot-capability subtree. Each compound task is represented after decomposition as an ordered composition of the corresponding primitive subtrees under a mission-level `execute_mission` sequence node. Task ordering is preserved through the sequence nodes. Task runtime-state preconditions, such as goal reception, door state, and sensor activation, are placed as condition nodes before the action nodes, and execution-time failures are handled by explicit recovery branches within the corresponding capability subtrees. Since feasibility preconditions are already resolved during HTN planning, infeasible and missing-target cases yield no plan and never reach the BT mapping stage. The BTs resulting from the mapping process are structurally valid, with a valid root node, valid control-flow nodes (sequence, fallback, and inverter), and registered, executable leaf nodes. The transformation was demonstrated on two example missions: a wheeled mobile manipulator executing a ground-floor door-opening route, and a legged manipulator traversing a staircase to the first floor. Both produced structurally valid trees that were exported to BehaviorTree.CPP v4 XML. As the two routing missions do not exercise sensing, the inspection and reporting subtrees were validated separately against full inspection plans, in which the capture and record operators mapped successfully to the respective visual, thermal, and depth inspection subtrees. The validation in this work remains structural and does not include execution on physical robots. The underlying subtree designs, however, follow patterns whose functionality has been validated on real robots in prior work [1].

Overall, the validation demonstrates the feasibility of the proposed BIM-to-HTN-to-BT pipeline. Information extraction from the BIM KG achieves strong retrieval performance, with the qwen2.5-coder 14B model passing 96.2% of the 53 benchmark queries (83.0% for the 3B model), where higher model capacity mainly improves Cypher validity and coordinate completeness. HTN-based task decomposition is deterministic and reproducible, with all representative missions satisfying completeness, correctness, ordering, and grounding, and infeasible cases reported explicitly rather than converted into unsupported robot actions. HTN-to-BT transformation produces structurally valid BTs that preserve task dependencies and execution logic and export to BehaviorTree.CPP v4 XML. The main limitations of the framework are the incomplete attribute-level information in the KG and the absence of robot-in-the-loop validation. Future work should therefore focus on richer BIM graph rep-

resentations, end-to-end testing of the framework pipeline and validation through execution in robotic environments.

IV. SUMMARY AND CONCLUSIONS

This paper has presented a graphRAG-based framework for automated task planning that transforms BIM-derived semantic and spatial information into robot-executable control structures for robotic inspection workflows. The proposed framework leverages BIM KGs, graph-based retrieval, LLMs, HTNs, and BTs within a unified planning and execution pipeline. BIM models are transformed into machine-interpretable KGs that encode building elements, spatial structures, and inspection-relevant relationships. Based on task queries, a graphRAG-based mechanism, in which an LLM translates natural-language task queries into graph queries, retrieves task-relevant semantic and spatial context, which is then used by a rule-based planner to generate HTN-based task plans. The generated HTN structures are transformed into BT-based execution models, enabling the generation of executable robotic inspection behaviors from BIM-derived knowledge. Overall, the contribution advances digital inspection, engineering informatics, and product-process modeling in the built environment. The results indicate the feasibility of combining BIM KGs, graph-based retrieval, and rule-based HTN task decomposition within a unified robotic task-planning framework. However, several limitations remain. For example, the framework currently supports only a limited set of tasks due to the partially defined HTN method library, which constrains generalizability across broader construction and inspection scenarios. Furthermore, task generation is highly dependent on the completeness and quality of the BIM-derived KG and the retrieved semantic and spatial context. The current graphRAG process also simplifies semantic reasoning and does not yet incorporate execution-relevant constraints, such as motion feasibility, dynamic obstacles, sensor uncertainty, and real-time changes in dynamic environments. Future work may focus on extending the HTN method library, improving graph-grounded retrieval and reasoning capabilities, integrating execution-relevant planning constraints, and validating the framework across additional BIM models, inspection scenarios, and real-world execution with robotic platforms.

ACKNOWLEDGMENTS

This study is a joint work conducted as part of research projects funded by the German Research Foundation (DFG) under grants SM 281/30-1, SM 281/31-1, SM 281/32-1, and SM 281/44-1. The financial support provided by DFG is gratefully acknowledged. Any opinions, findings, conclusions, or recommendations expressed in this paper are those of the authors and do not necessarily reflect the views of DFG.

REFERENCES

[1] A. Tandon, J. Stührenberg, and K. Smarsly, "Automated building inspections coupling building information modeling and behavior trees," in *Proc. European Conference on Computing in Construction and CIB W78 Conference on IT in Construction*, Porto, Portugal, Jul. 2025, doi: 10.35490/EC3.2025.360.

[2] K. Smarsly, M. Theiler, and K. Dragos, "IFC-based modeling of cyber-physical systems in civil engineering," in *Proc. 24th EG-ICE International Workshop on Intelligent Computing in Engineering*, 2017, pp. 269–278.

[3] M. Theiler and K. Smarsly, "IFC Monitor – An IFC schema extension for modeling structural health monitoring systems," *Advanced Engineering Informatics*, vol. 37, pp. 54–65, 2018, doi: 10.1016/j.aei.2018.04.011.

[4] W. Yang, L. Some, M. Bain, and B. Kang, "A comprehensive survey on integrating large language models with knowledge-based methods," *Knowledge-Based Systems*, vol. 318, article no. 113503, 2025, doi: 10.1016/j.knosys.2025.113503.

[5] H. Yue, Q. Wang, Z. Zhao, S. Lai, and G. Huang, "Interactions between BIM and robotics: Towards intelligent construction engineering and management," *Computers in Industry*, vol. 169, article no. 104299, 2025, doi: 10.1016/j.compind.2025.104299.

[6] T. Fitz, M. Theiler, and K. Smarsly, "A metamodel for cyber-physical systems," *Advanced Engineering Informatics*, vol. 41, article no. 100930, 2019, doi: 10.1016/j.aei.2019.100930.

[7] J. Stührenberg and K. Smarsly, "LIO-BIM – Coupling lidar inertial odometry with building information modeling for robot localization and mapping," *Advanced Engineering Informatics*, vol. 66, article no. 103477, 2025, doi: 10.1016/j.aei.2025.103477.

[8] I. Georgievski and M. Aiello, "HTN planning: Overview, comparison, and beyond," *Artificial Intelligence*, vol. 222, pp. 124–156, 2015, doi: 10.1016/j.artint.2015.02.002.

[9] M. Iovino, E. Scukins, J. Styrud, P. Ögren, and C. Smith, "A survey of behavior trees in robotics and AI," *Robotics and Autonomous Systems*, vol. 154, article no. 104096, 2022, doi: 10.1016/j.robot.2022.104096.

[10] M. G. Ozsoy, L. Messallem, J. Besga, and G. Minneci, "Text2Cypher: Bridging natural language and graph databases," arXiv preprint arXiv:2412.10064, 2024, doi: 10.48550/arXiv.2412.10064.

[11] J. Zhu, P. Wu, and X. Lei, "IFC-graph for facilitating building information access and query," *Automation in Construction*, vol. 148, article no. 104778, 2023, doi: 10.1016/j.autcon.2023.104778.

[12] Neo4j, Inc., *Neo4j Graph Database*. Available: `https://neo4j.com`. Accessed: Jun. 12, 2026.

[13] S. Hellin, S. Jang, S. Fuchs, S. Nousias, and A. Borrmann, "BIM information extraction through LLM-based adaptive exploration," arXiv preprint arXiv:2605.01698, 2026, doi: 10.48550/arXiv.2605.01698.

[14] Ollama, "Qwen2.5-Coder," Ollama model library. [Online]. Available: https://ollama.com/library/qwen2.5-coder. Accessed: Jun. 10, 2026.